

Week 07 Explained Student Worksheet

Term Frequency Document Frequency and TF-IDF

Name _____ Student ID _____ Group _____

Replace binary occurrence values with numerical term weights. Calculate a small example by hand, then compute a complete TF-IDF matrix for the Week 6 collection.

Use this worksheet during the two-hour practical session. Read, predict, run, and explain. The existing lab sheet contains the complete program; this companion explains key ideas and provides space to record your own evidence. Python 3.10 or later is sufficient; no pip packages are needed.

What you should be able to do

- Count term frequencies within individual documents.
- Count document frequencies across the collection.
- Calculate natural-log inverse document frequency.
- Construct and interpret TF-IDF vectors without external packages.

Understand the concepts

$TF(t,d)$ is the number of occurrences of term t in document d after preprocessing. $DF(t)$ counts documents containing t , not token occurrences. N is the number of loaded documents, including a document whose preprocessing yields no tokens.

Use $IDF(t) = \log(N / DF(t))$, with `math.log` providing the natural logarithm. $TF-IDF(t,d) = TF(t,d) \times IDF(t)$. This lab uses raw counts, no smoothing, and no separate vector normalization. A term in every document has IDF zero; a rarer term receives a larger IDF.

Only vocabulary terms have an IDF entry, so each computed DF is at least one. An unknown term has no vocabulary coordinate and no $\log(N/0)$ is calculated. An empty collection returns empty structures. A document with no retained tokens has an all-zero vector.

All document vectors share one sorted vocabulary. The i -th coordinate always represents `vocabulary[i]`. Keep this coordinate order with the vectors so dot products in Week 8 compare matching terms.

Predict before running

In three documents, `data` occurs twice in `d1` and once in `d2`. State $TF(data,d1)$ and $DF(data)$ separately.

Prepare your terminal

Extract the course and open a terminal inside Week_07_TF_IDF. Keep docs and data beside the Python files. The main demonstration is lab07_solution.py; your exercise starter is lab07_exercises.py. Starters run but deliberately contain unfinished TODO functions.

```
py -3.13 lab07_solution.py
py -3.13 lab07_exercises.py
```

Use py -3 or python instead if that is your working Python 3.10+ command.

Worked example with explanation

Hand calculation first: let $d1 = \text{"data data python"}$, $d2 = \text{"data java"}$, and $d3 = \text{"java"}$. With $N = 3$, calculate TF, DF, IDF, and every nonzero TF-IDF entry before running the check below.

```
import math
documents = {"d1": ["data", "data", "python"],
             "d2": ["data", "java"], "d3": ["java"]}
for term in ["data", "java", "python"]:
    df = sum(term in tokens for tokens in documents.values())
    idf = math.log(len(documents) / df)
    weights = [tokens.count(term) * idf
               for tokens in documents.values()]
    print(term, df, round(idf, 4),
          [round(value, 4) for value in weights])
```

Expected output from an executed example

```
data 2 0.4055 [0.8109, 0.4055, 0.0]
java 2 0.4055 [0.0, 0.4055, 0.4055]
python 1 1.0986 [1.0986, 0.0, 0.0]
```

How each stage produces the result

1. The example has three documents, so N is 3. DF counts documents containing a term, not the number of times that term appears.
2. math.log uses the natural logarithm. With $DF=2$, the IDF calculation is $\text{math.log}(3 / 2)$, approximately 0.4055.
3. The data TF in $d1$ is 2, so its weight is twice the unrounded IDF, approximately 0.8109. python occurs in one document, giving IDF approximately 1.0986.
4. This course uses raw TF and unsmoothed IDF. Round for display only. If a retained term appears in every document, its IDF is zero and its TF-IDF weights are zero.

Explain one intermediate value in your own words

Read the weekly program

The following excerpt is copied exactly from lab07_solution.py. It depends on the imports, constants, and helper functions in that file. Read it there rather than pasting it into an empty script and expecting the excerpt to run alone.

```
def build_tfidf(documents):
    """Raw TF and natural-log IDF; no smoothing or extra normalization."""
    counts = {}
    document_frequency = {}
    for name, text in documents.items():
        counts[name] = term_frequency(tokenize(text))
        for term in counts[name]:
            document_frequency[term] = document_frequency.get(term, 0) + 1
    vocabulary = sorted(document_frequency)
    idf = {}
    total = len(documents)
    for term in vocabulary:
        idf[term] = math.log(total / document_frequency[term])
    vectors = {}
    for name in sorted(documents):
        vectors[name] = [
            counts[name].get(term, 0) * idf[term] for term in vocabulary
        ]
    return vocabulary, counts, document_frequency, idf, vectors
```

Code walkthrough

1. counts[name] is a frequency dictionary built from the full token list. Iterating its keys visits each distinct term once for that document.
2. document_frequency is incremented once per document containing a term. This keeps every known DF between 1 and N.
3. sorted(document_frequency) defines one vocabulary order. IDF is calculated only for terms present in that vocabulary.
4. For every document, the comprehension places its TF times the collection IDF at the same term coordinate. A missing term has TF zero. The function returns vocabulary, counts, DF, IDF, and vectors for later inspection.

If data is repeated in one existing document, which TF values change? Does DF change if that document already contained data?

Connection to the course

Week 8 builds the query vector using this same vocabulary and collection IDF. The vectors are useful only when their coordinates describe the same terms.

Guided practical activity

Work through the steps in order. Compare your predictions with observations and record at least one specific value per experiment. Keep original baseline data until you have saved the first transcript. Put experiments that change files in a disposable copy.

Step 1 Complete the three-document hand calculation above before opening the solution.

Use the small example to check the formula before debugging the larger matrix.

Step 2 Load exactly the ten documents from Week 6 and tokenize each one.

Keep the ten-document Week 6 collection unchanged for the initial comparison.

Step 3 Count TF values using a dictionary, preserving repeated tokens.

Count every retained occurrence for TF. Replacing tokens with a set here would lose frequency.

Step 4 For each document, increment DF once for each distinct term in that document.

Iterate distinct keys for DF. Compare the counts with lengths of posting lists.

Step 5 Calculate IDF with math.log and construct vectors using one sorted vocabulary.

Use one sorted vocabulary for all document vectors. Do not independently sort coordinates within each vector.

Step 6 Print the full matrix. It appears in blocks of five document columns so it remains readable. Compare selected entries with independent hand calculations.

Check selected cells using TF, DF, and N from the same collection. Printed values are rounded, while stored values retain precision.

Test record

Check to demonstrate	Actual observation	Pass or revise
Known terms have $1 \leq DF \leq N$		
Posting lengths agree with computed DF		
A term in every document has zero IDF		

Record one prediction that changed after running the program and explain why

Independent work and submission

Complete these selected tasks in your own exercise file. Use the existing lab sheet for additional exercises. Do not submit the unchanged starter or simply import a solution in place of implementing the requested function.

- Hand calculation: add `d4 = "data"` to the worked example and recalculate `DF(data)`, `IDF(data)`, and the `d1` data weight.
- Implementation: finish `tfidf_weight` in the starter for known terms with $1 \leq DF \leq N$.
- Debugging: explain why incrementing `DF` for every token occurrence can produce a negative `IDF`.
- Interpretation: add the same retained term to every document and explain its resulting `IDF`.

Diagnose common errors

IDF becomes negative for a known term. `DF` was probably counted per token occurrence. Count each distinct term once per document.

Hand calculations differ after rounding. Retain full precision during computation. Round only the final printed values.

Submit these items

- Your complete hand calculation and the completed weighting function.
- The generated matrix and independent checks for at least three entries.
- An explanation of the effects of repetition and collection-wide common terms.

Your evidence should identify the input or query, expected result, actual result, and explanation. State any document edits or preprocessing changes so the result can be reproduced. Include this completed worksheet or a clearly numbered digital response.

Exit questions

How do `TF` and `DF` differ?

Which logarithm does `math.log` use?

Optional challenge

Implement log-scaled `TF` as a separate experiment, leaving the required raw-`TF` baseline unchanged. Compare the resulting weights for a term repeated once, twice, and ten times.

Source for this companion

Based on the supplied Week 07 lab sheet, shipped program, exercise starter, and sample data. The complete main program and `expected_output.txt` in this folder are the references for checking your baseline run.